Cut Cross-Entropy

Вычисление функции потерь без полной матрицы логитов

Wijmans et al., arXiv:2411.09009v2

Метод и результаты статьи

Откуда берётся расход памяти

МАТРИЦА ЛОГИТОВ

На выходе модели для N позиций вычисляют логиты по всему словарю |V|. Получается матрица размером N × |V|.

8 192 × 256 000

позиций × элементов словаря Gemma 2 2B

Такая матрица в FP32 занимает около 8,4 ГБ. У CCE дополнительная память растёт как O(N + |V|), а не O(N × |V|).

Разложение кросс-энтропии

Для каждой позиции функция потерь состоит из двух слагаемых:

$$\ell_i = \log \sum_{j=1}^{|V|} \exp(e_i^\top c_j) - e_i^\top c_{y_i}$$

ЛОГИТ ЦЕЛЕВОГО ТОКЕНА

Его вычисляют отдельно, по известному индексу токена.

СУММА ПО СЛОВАРЮ

Log-sum-exp считают по блокам словаря.

В статье максимизируют log softmax; здесь показана функция потерь −log softmax.

Прямой проход без полной матрицы логитов

1. БЛОКИ

Triton kernel загружает блоки скрытых состояний и весов выходного слоя.

2. LSE

Вычисляет логиты блока и сразу обновляет log-sum-exp.

3. ПАМЯТЬ

Промежуточные логиты остаются на чипе GPU (SRAM). Полную матрицу $N \times |V|$ не записывают в видеопамять.

Отдельная оптимизация: позиции с игнорируемой меткой исключают до расчёта логитов.

Обратный проход: фильтрация малых вкладов

Производная по логиту равна вероятности класса минус 1 для целевого токена:

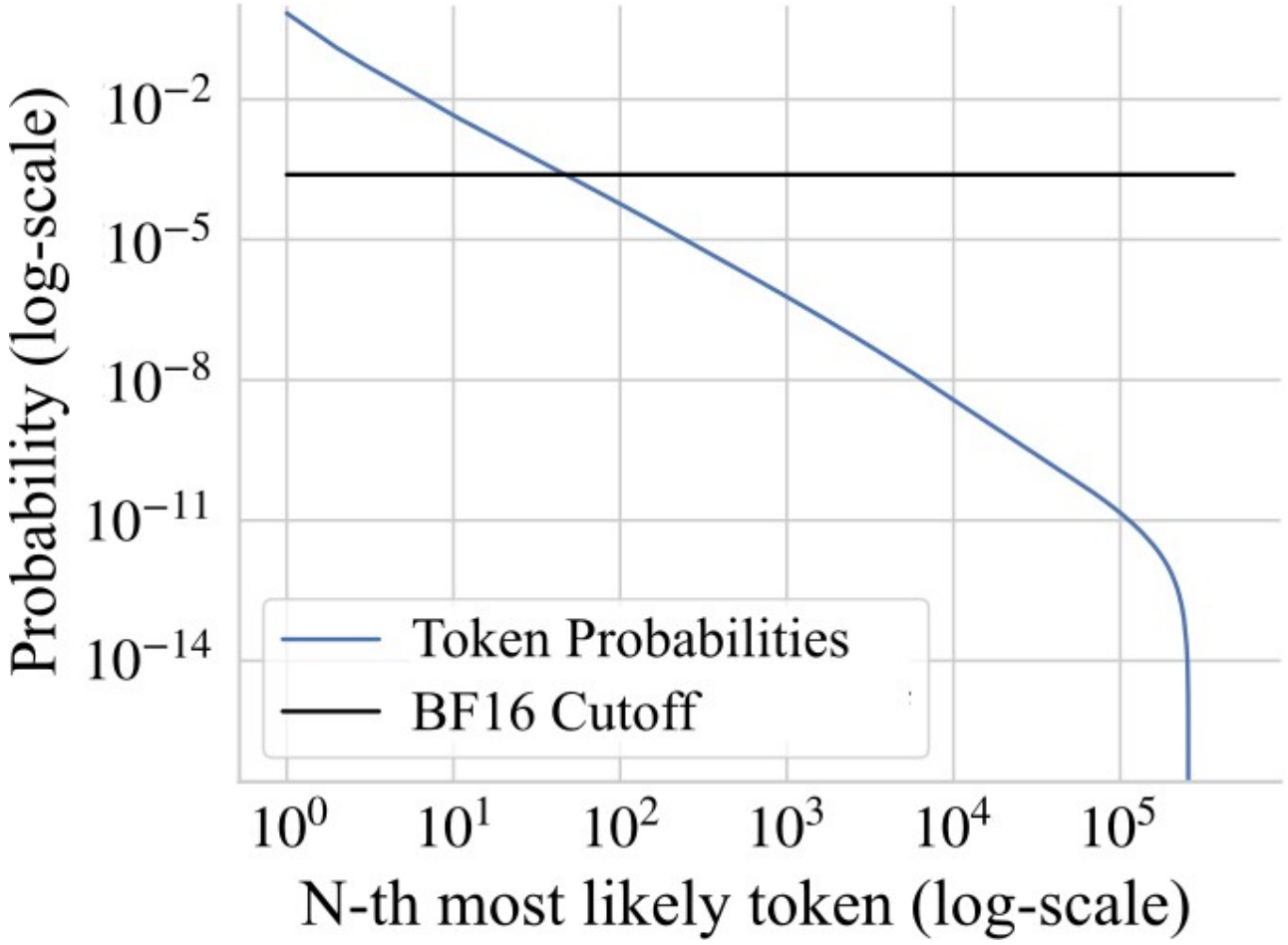
$$\frac{\partial \ell_i}{\partial z_{ij}} = p_{ij} - \mathbf{1}[j = y_i]$$

$$\varepsilon = 2^{-12}$$

Если все вероятности в блоке ниже порога, kernel не считает его вклад в градиент.

Чтобы чаще пропускать блоки целиком, авторы сортируют словарь по среднему логиту.

Почему можно пропускать блоки



Источник рисунка: Wijmans et al., Figure 3

РИСУНОК 3

График показывает среднюю вероятность токена в зависимости от его места в ранжировании.

ПОРОГ ФИЛЬТРАЦИИ

Примерно с 50-го места средняя вероятность ниже 2^{-12}. В BF16 такие значения часто теряются при округлении или суммировании.

Kernel пропускает блок, если все вероятности в нём ниже порога.

BF16 хранит и меньшие числа: дело в точности вычисления.

Зачем сортировать словарь

ПРОБЛЕМА

Пропустить можно лишь блок, в котором все вероятности малы. Если хотя бы одна выше порога, блок нужно обработать.

РЕШЕНИЕ

В прямом проходе считают средний логит каждого элемента словаря. При обратном проходе элементы с близкими значениями группируют.

ИЗМЕРЕНИЕ

Без сортировки функция потерь и градиент считаются 159 мс вместо 145 мс на Gemma 2 2B.

Сортировка не удаляет токены из словаря.

Память и время: Gemma 2 2B на A100

Функция потерь и градиент, 8 192 позиции

PYTORCH

28 000 МБ

208 мс

CCE

1 164 МБ

145 мс

Если считать только функцию потерь, пик памяти составляет 1 МБ у CCE против 24 000 МБ у PyTorch.

torch.compile: 16 000 МБ и 143 мс. На Phi 3.5 Mini CCE примерно на 50% медленнее: приложение C.

Фильтрация в обратном проходе

Gemma 2 2B на A100, функция потерь и градиент

ССЕ С ФИЛЬТРАЦИЕЙ

145 мс

БЕЗ ФИЛЬТРАЦИИ

357 мс

Без фильтрации обратный проход занимает 314 мс вместо 100 мс.

Главную экономию времени даёт пропуск блоков с малыми вероятностями.

Влияние на обучение

ДООБУЧЕНИЕ

Кривые функции потерь для CCE и torch.compile почти совпали на четырёх моделях.

ПРЕДОБУЧЕНИЕ

Фильтрация может обнулить градиенты редких токенов. Поэтому для весов классификатора её отключают, а ошибку BF16 снижают суммированием Кахана.

ВАРИАНТ ДЛЯ ПРЕДОБУЧЕНИЯ

Для CCE-Kahan-FullC и torch.compile кривые перплексии совпали. На Mistral NeMo удвоенный батч сократил обучение на 2 ч (16%), хотя расчёт потерь и градиентов стал медленнее.

LM head и баланс стадий PP

КОНВЕЙЕРНЫЙ ПАРАЛЛЕЛИЗМ (PP)

PP работает лучше, когда вычислительная нагрузка стадий близка. Их проще балансировать, если блоки похожи по отношению памяти к вычислениям.

LM HEAD ТРЕБУЕТ БОЛЬШЕ ПАМЯТИ

По сравнению с другими блоками модели LM head расходует непропорционально много памяти на единицу вычислений.

ВОЗМОЖНЫЙ ЭФФЕКТ CCE

Снижение расхода памяти может упростить балансировку PP или позволить обойтись меньшим числом стадий.

Это предположение авторов: замеров с PP в статье нет.